HAIL-DSS - Complete Project Documentation

Table of Contents

Project Overview

HAIL-DSS (Human-AI Life Decision Support System) is a web-based application that helps users make informed decisions across multiple life domains including career, education, finance, and lifestyle.

Core Purpose

- Provide AI-powered decision analysis
- Evaluate options based on multiple weighted dimensions
- Offer transparent, explainable recommendations with confidence scores
- Maintain privacy by storing all data locally

Key Features

- **Multi-Criteria Decision Analysis**: Structured evaluation across 8+ dimensions
- **AI-Powered**: Uses Google Gemini for intelligent decision analysis
- **Fallback System**: Rule-based engine when AI is unavailable
- **Privacy-First**: All data stored locally in browser
- **Input Validation**: AI validates user inputs to reject nonsensical entries

---

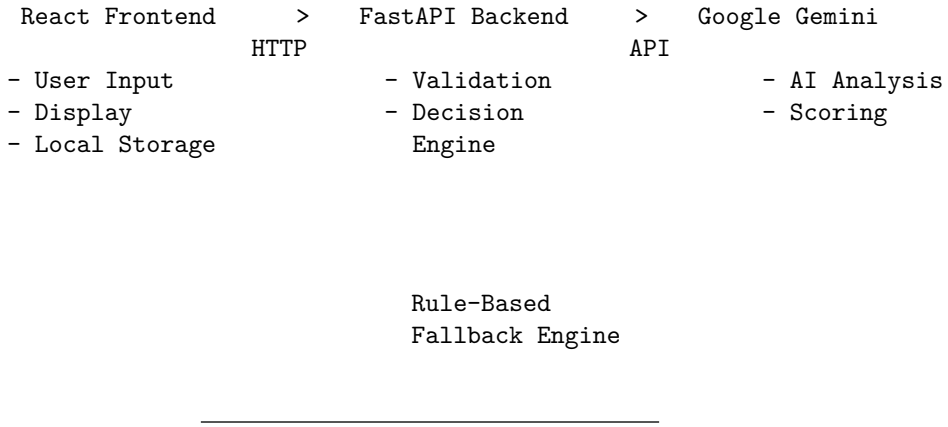
System Architecture

Technology Stack

Backend: - **Framework**: FastAPI (Python) - **AI Engine**: Google Gemini API - **Validation**: Pydantic models - **CORS**: Enabled for frontend communication

Frontend: - **Framework**: React 18 with Vite - **State Management**: React Context API - **HTTP Client**: Axios - **Charts**: Recharts - **Styling**: Custom CSS with CSS variables

Architecture Diagram



How It Works

User Decision Flow

1. **Goal Selection**: User chooses decision category (career, education, finance, lifestyle)
2. **Options Input**: User enters 2-5 options to compare
3. **Constraints**: User sets budget, time, and risk parameters
4. **Priorities**: User selects key decision factors
5. **AI Validation**: Gemini validates that options are meaningful
6. **AI Analysis**: Gemini evaluates each option across multiple dimensions
7. **Scoring**: System calculates weighted scores and confidence levels
8. **Results**: User sees ranked recommendations with detailed analysis
9. **Local Storage**: Decision is saved to browser's local storage

Decision Categories

Each category has specific evaluation dimensions:

Career: - Financial feasibility - Growth potential - Risk alignment - Time feasibility - Passion alignment

Education: - Affordability - Reputation/quality - Duration feasibility - Employability - Learning fit

Finance: - Expected returns - Risk alignment - Liquidity - Time feasibility - Effort/simplicity

Lifestyle: - Wellbeing impact - Financial feasibility - Time feasibility - Social impact - Risk alignment

---

Backend Code Structure

Directory Structure

```
backend/
  server.py              # Main FastAPI application
  .env                   # Environment variables (GEMINI_API_KEY)
  requirements.txt       # Python dependencies
  engine/
      __init__.py
      decision_engine.py    # Core decision orchestration
      ai_engine.py          # Gemini AI integration
      career_rules.py       # Rule-based career evaluation
      education_rules.py    # Rule-based education evaluation
      finance_rules.py      # Rule-based finance evaluation
      lifestyle_rules.py    # Rule-based lifestyle evaluation
```

server.py - Main Application

Purpose: FastAPI server that handles HTTP requests and orchestrates decision analysis.

Key Components:

1. **CORS Middleware**: Allows frontend (localhost:5173) to communicate

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173", "http://127.0.0.1:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

2. **Request Model (DecisionRequest)**: Validates incoming decision data

```
class DecisionRequest(BaseModel):
    goal: Literal["career", "education", "finance", "lifestyle"]
    options: list[str]  # 2-5 options
    budget: float
    risk_tolerance: Literal["low", "medium", "high"]
    time_months: int
    priorities: list[str]
    context: dict
```

3. **Response Model (DecisionResponse)**: Structured decision results

```
class DecisionResponse(BaseModel):
    goal: str
    ranked_paths: list[dict]
    summary: str
```

4. **Main Endpoint (/api/decide)**: Processes decision requests

```
@app.post("/api/decide", response_model=DecisionResponse)
async def decide(req: DecisionRequest):
    ranked = run_decision_engine(
        goal=req.goal,
        options=req.options,
        budget=req.budget,
        risk_tolerance=req.risk_tolerance,
        time_months=req.time_months,
        priorities=req.priorities,
        context=req.context,
    )
    return DecisionResponse(goal=req.goal, ranked_paths=ranked, summary=summary)
```

decision_engine.py - Core Orchestration

Purpose: Orchestrates AI evaluation with fallback to rule-based modules.

Key Functions:

1. **run_decision_engine()**: Main entry point
   - Validates inputs via AI
   - Attempts AI evaluation (Gemini)
   - Falls back to rule-based engine if AI fails
   - Ranks results by score
   - Returns structured decision paths
2. **_compute_weighted_score()**: Calculates final score from dimension scores

```
def _compute_weighted_score(dimension_scores: dict, weights: dict) -> float:
    total = 0.0
    weight_sum = sum(weights.values())
    for dim, weight in weights.items():
        total += dimension_scores.get(dim, 0.5) * (weight / weight_sum)
    return round(total, 4)
```

3. **_compute_confidence()**: Calculates confidence based on pros/cons balance

```
def _compute_confidence(score: float, pros: list, cons: list) -> float:
    balance = len(pros) / max(len(pros) + len(cons), 1)
```

```
    confidence = score * 0.7 + balance * 0.3
    return round(min(confidence, 1.0), 4)
```

ai_engine.py - Gemini Integration

Purpose: Integrates Google Gemini AI for intelligent decision analysis.

Key Components:

1. **GOAL_DIMENSIONS**: Defines evaluation dimensions for each goal type

```
GOAL_DIMENSIONS = {
    "career": ["financial_feasibility", "growth_potential", "risk_alignment", "time_feasibil
    "education": ["affordability", "reputation_quality", "duration_feasibility", "employabil
    "finance": ["expected_returns", "risk_alignment", "liquidity", "time_feasibility", "effo
    "lifestyle": ["wellbeing_impact", "financial_feasibility", "time_feasibility", "social_i
}
```

2. **validate_options()**: AI-powered input validation
   - Sends options to Gemini with validation prompt
   - Checks if options are legitimate, meaningful choices
   - Rejects gibberish, random characters, profanity
   - Raises ValueError if invalid
3. **evaluate_options_with_ai()**: AI-powered decision evaluation
   - Constructs detailed prompt with user context
   - Sends to Gemini for analysis
   - Parses JSON response with dimension scores
   - Calculates weighted scores and confidence
   - Returns structured results

Prompt Engineering: The system uses carefully crafted prompts to ensure: - Specific, analytical scoring (0.0 to 1.0 per dimension) - Concrete pros and cons (2-4 pros, 1-3 cons) - 2-3 sentence explanations - Strict JSON output format

Rule-Based Modules (career_rules.py, education_rules.py, etc.)

Purpose: Provide fallback evaluation when AI is unavailable.

Structure: Each module contains: - **WEIGHTS dictionary**: Importance weights for each dimension - **evaluate_*_option() function**: Rule-based scoring logic - Keyword matching and heuristic analysis

Example (career_rules.py):

```
CAREER_WEIGHTS = {
    "financial_feasibility": 0.25,
    "growth_potential": 0.25,
    "risk_alignment": 0.20,
    "time_feasibility": 0.15,
```

```
    "passion_alignment": 0.15,
}

def evaluate_career_option(option, budget, risk_tolerance, time_months, priorities, context)
    # Keyword-based analysis
    # Returns dimension scores, pros, cons, explanation
```

Frontend Code Structure

Directory Structure

```
frontend/
  src/
      main.jsx              # React entry point
      App.jsx               # Main application component
      index.css             # Global styles
      components/
          Navbar.jsx        # Navigation bar
          Dashboard.jsx     # Landing page
          DecisionForm.jsx  # Multi-step decision input form
          ResultsPanel.jsx  # Results display with charts
          HistoryPanel.jsx  # Decision history sidebar
      contexts/
          LanguageContext.jsx  # Language state management
      hooks/
          useTranslation.js   # Translation hook
          useLiveTranslation.js # Live AI translation
      services/
          decisionService.js   # API communication
      utils/
           storage.js        # Local storage utilities
           translation.js    # Translation utilities
  package.json
  vite.config.js
```

App.jsx - Main Application

Purpose: Root component managing application state and view routing.

State Management:

```
const [view, setView] = useState(VIEWS.DASHBOARD)  // Current view
const [result, setResult] = useState(null)          // Decision results
const [loading, setLoading] = useState(false)       // Loading state
const [error, setError] = useState(null)            // Error state
```

```
const [historyOpen, setHistoryOpen] = useState(false)  // History panel
const [history, setHistory] = useState(loadHistory)    // Decision history
```

View States: - **DASHBOARD**: Landing page with "Start Decision" button - **FORM**: Multi-step decision input form - **RESULTS**: Ranked results with charts and analysis

Key Functions:

1. **handleSubmit()**: Processes decision form

```
const handleSubmit = async formData => {
    setLoading(true)
    const data = await getDecision(formData)  // API call
    saveDecision(data)  // Local storage
    setResult(data)
    setView(VIEWS.RESULTS)
}
```

2. **Error Handling**: Distinguishes between validation errors (400) and system errors

DecisionForm.jsx - Input Form

Purpose: Multi-step form for collecting decision parameters.

Form Steps: 1. **Goal Selection**: Choose category (career, education, finance, lifestyle) 2. **Options Input**: Add 2-5 options to compare 3. **Constraints**: Budget, time, risk tolerance 4. **Priorities**: Select key factors (optional)

Features: - Dynamic option addition/removal - Input validation - Progress indicator - Responsive design

ResultsPanel.jsx - Results Display

Purpose: Displays ranked decision results with visualizations.

Components: - **Ranked Options**: Sorted list with scores and confidence - **Dimension Charts**: Radar/bar charts showing dimension breakdowns - **Pros/Cons**: Bullet points for each option - **Explanation**: AI-generated analysis - **Comparison**: Side-by-side option comparison

Charts: Uses Recharts library for interactive visualizations

decisionService.js - API Communication

Purpose: Handles HTTP communication with backend.

Key Function:

```
export const getDecision = async (formData) => {
    const response = await axios.post('http://localhost:8000/api/decide', formData)
```

```
    return response.data
}
```

storage.js - Local Storage

Purpose: Manages browser local storage for decision history.

Functions: - **saveDecision()**: Stores decision with timestamp - **loadHistory()**: Retrieves all saved decisions - **deleteDecision()**: Removes specific decision

Privacy: All data stored locally, never transmitted to server

---

Key Components Explained

1. Decision Engine (Backend)

Location: `backend/engine/decision_engine.py`

Responsibility: Core decision analysis orchestration

How It Works: 1. Receives decision parameters from API 2. Validates inputs using AI 3. Attempts AI evaluation via Gemini 4. Falls back to rule-based engine if AI fails 5. Computes weighted scores and confidence 6. Ranks options by score 7. Returns structured results

Key Insight: Hybrid approach ensures reliability - AI provides intelligent analysis, rules provide backup

2. AI Engine (Backend)

Location: `backend/engine/ai_engine.py`

Responsibility: Gemini AI integration for validation and analysis

How It Works: 1. Initializes Gemini client with API key 2. Constructs detailed prompts with user context 3. Sends requests to Gemini API 4. Parses JSON responses 5. Validates and clamps scores to [0, 1] 6. Calculates confidence metrics

Key Insight: Prompt engineering ensures consistent, structured AI outputs

3. Rule-Based Modules (Backend)

Location: `backend/engine/career_rules.py`, `education_rules.py`, etc.

Responsibility: Fallback evaluation when AI is unavailable

How It Works: 1. Uses keyword matching and heuristics 2. Applies predefined weights to dimensions 3. Generates pros/cons based on patterns 4. Provides basic explanations

Key Insight: Ensures system functionality even without AI access

4. App Component (Frontend)

Location: `frontend/src/App.jsx`

Responsibility: Application state management and routing

How It Works: 1. Manages view state (dashboard, form, results) 2. Handles form submission and API calls 3. Manages error states and loading 4. Coordinates with history panel 5. Provides error toasts for user feedback

Key Insight: Central state management simplifies component communication

5. Decision Form (Frontend)

Location: `frontend/src/components/DecisionForm.jsx`

Responsibility: Collect user decision parameters

How It Works: 1. Multi-step form with validation 2. Dynamic option fields (add/remove) 3. Context-aware input hints 4. Progress tracking 5. Form submission to API

Key Insight: User-friendly input collection with validation

Data Flow

Complete Request Flow

```
1. User fills DecisionForm
   ↓
2. Frontend validates form
   ↓
3. Frontend calls decisionService.getDecision()
   ↓
4. Axios POST to http://localhost:8000/api/decide
   ↓
5. FastAPI receives request
   ↓
6. Pydantic validates request model
   ↓
7. server.py calls run_decision_engine()
   ↓
8. decision_engine.py validates options via AI
   ↓
9. ai_engine.py sends validation prompt to Gemini
   ↓
```

```
10. Gemini returns validation result
   ↓
11. If valid, ai_engine.py sends evaluation prompt to Gemini
   ↓
12. Gemini returns dimension scores, pros, cons, explanations
   ↓
13. decision_engine.py computes weighted scores and confidence
   ↓
14. Results ranked by score
   ↓
15. server.py returns DecisionResponse
   ↓
16. Frontend receives response
   ↓
17. Results saved to local storage
   ↓
18. View changes to RESULTS
   ↓
19. ResultsPanel displays ranked options with charts
```

Error Flow

```
AI Validation Fails
→ ValueError raised
→ HTTP 400 returned
→ Frontend shows validation error toast

AI Evaluation Fails
→ Exception caught
→ Fallback to rule-based engine
→ Results computed with rules
→ Note added to explanation

API Error
→ Exception caught
→ HTTP 500 returned
→ Frontend shows generic error toast
```

AI Integration

Google Gemini API

Model: gemini-2.5-flash

API Key: Stored in `backend/.env` file as `GEMINI_API_KEY`

Two AI Operations:

1. **Input Validation**
   - Purpose: Reject nonsensical inputs
   - Prompt: Checks if options are legitimate choices
   - Output: JSON with validity flag and invalid options list
2. **Decision Evaluation**
   - Purpose: Analyze and score options
   - Prompt: Evaluates options across dimensions with user context
   - Output: JSON with dimension scores, pros, cons, explanations

Prompt Engineering Strategy

Validation Prompt: - Strict JSON output requirement - Clear criteria for valid vs invalid - Examples of valid options - Rejects gibberish, profanity, unrelated content

Evaluation Prompt: - Includes all user context (budget, risk, time, priorities) - Lists specific dimensions for the goal type - Specifies score range (0.0 to 1.0) - Requires concrete pros/cons - Demands 2-3 sentence explanations - Strict JSON output format

Fallback Strategy

If Gemini API fails: 1. Exception caught in decision_engine.py 2. Warning logged to console 3. Rule-based engine activated 4. Results computed with heuristics 5. Note added: "[Note: Generated by rule-based fallback engine]"

Running the Project

Prerequisites

- Python 3.8+
- Node.js 16+
- Google Gemini API key

Backend Setup

1. **Navigate to backend directory**:

```
cd backend
```

2. **Install dependencies**:

```
pip install -r requirements.txt
```

3. **Set up environment variables**: Create `.env` file:

```
GEMINI_API_KEY=your_api_key_here
```

4. **Run the server**:

```
python server.py
```

Server runs on `http://localhost:8000`

Frontend Setup

1. **Navigate to frontend directory**:

```
cd frontend
```

2. **Install dependencies**:

```
npm install
```

3. **Run the development server**:

```
npm run dev
```

Frontend runs on `http://localhost:5173`

Testing the System

1. Open browser to `http://localhost:5173`
2. Click "Start Decision"
3. Select a goal (e.g., "career")
4. Enter 2-5 options (e.g., "Software Engineer", "Data Scientist")
5. Set constraints (budget, time, risk)
6. Submit and view AI-powered results

Technical Details

Backend Dependencies

FastAPI: Modern, fast web framework for building APIs - Automatic validation with Pydantic - Async support - OpenAPI documentation

google-genai: Google Gemini AI SDK - AI model access - Content generation - JSON parsing

python-dotenv: Environment variable management - Load API keys from .env - Security best practices

uvicorn: ASGI server - Production-ready server - Hot reload in development

Frontend Dependencies

React 18: UI library - Component-based architecture - Hooks for state management - Virtual DOM for performance

Vite: Build tool - Fast HMR (Hot Module Replacement) - Optimized production builds - Modern ES modules

Axios: HTTP client - Promise-based API calls - Request/response interceptors - Error handling

Recharts: Charting library - Declarative API - Responsive charts - Customizable styling

Data Models

Decision Request:

```
{
  goal: "career|education|finance|lifestyle",
  options: ["Option 1", "Option 2", ...],
  budget: number,
  risk_tolerance: "low|medium|high",
  time_months: number,
  priorities: ["priority1", "priority2", ...],
  context: {}
}
```

Decision Response:

```
{
  goal: string,
  ranked_paths: [
    {
      option: string,
      score: number (0-1),
      score_pct: number (0-100),
      confidence: number (0-1),
      confidence_pct: number (0-100),
      rank: number,
      pros: [string],
      cons: [string],
      dimension_scores: {
        dimension_name: number (0-100),
        ...
      },
      explanation: string
    }
  ],
  summary: string
}
```

Performance Considerations

Backend: - Async endpoints for non-blocking I/O - Connection pooling for AI API calls - Efficient JSON parsing - Minimal memory footprint

Frontend: - React.memo for component optimization - Lazy loading for charts - Debounced API calls - Efficient state updates

Security Features

Backend: - Input validation with Pydantic - CORS configuration - No data storage on server - Environment variables for secrets

Frontend: - XSS protection via React - Local storage for privacy - No external data transmission - Secure API communication

---

Conclusion

HAIL-DSS is a sophisticated decision support system that combines AI-powered analysis with rule-based fallbacks to provide reliable, transparent decision recommendations. The system prioritizes user privacy through local storage and ensures robustness through hybrid AI/rule architecture.

The modular codebase allows for easy extension to new decision categories and integration of additional AI models in the future.

Project Status: Production Ready
Version: 1.0.0
Last Updated: May 2025